

Introduction

College0 – AI-Enabled Online College Program Management System

Project	College0
Course	CSC 32200
Team	Team A
Members	Aung, Karim, Zihad, Kazi, Ferdosi

1. Introduction

1.1 Purpose

This design report presents the data structures, object interactions, and system logic for the College0 system. The purpose of this report is to serve as the implementation blueprint for the project. The report expands on the Software Requirements Specification by defining how the system will be organized internally and how each major function will be carried out.

Every class, entity, method, and diagram in this document is directly traceable to a requirement in the SRS. The implementation must not deviate from this document without a recorded team decision.

1.2 System Overview

College0 is a local GUI-based academic management system that supports four user roles: Visitor, Student, Instructor, and Registrar. The system manages the full semester cycle, including admission, semester setup, registration, class execution, reviews, grading, graduation, complaints, and AI-based assistance. The system is divided into six major subsystems, each implemented by one or more dedicated service classes:

User Management Subsystem – AuthService

Handles login, session management, role-based access control, and password enforcement.

Manages account creation upon admission approval (User + Student/Instructor records).

Generates student IDs and temporary passwords; flags first-login password-change requirement.

Stores VisitorApplication records and routes them to the Registrar for review.

Semester Management Subsystem – RegistrationService

Controls the semester phase state machine: Setup → Registration → Running → Grading → Finalized.

Enforces which operations are permitted in each phase (e.g., enrollment only during Registration).

Registrar triggers all phase transitions manually.

Course Management Subsystem – RegistrationService

Manages the Course catalog, ClassSection offerings, Enrollment records, and WaitlistEntry positions.

Enforces a 2–4 course load per student per semester.

Runs four sequential gates before confirming enrollment: phase check, load limit, conflict check, capacity check.

Creates WaitlistEntry if the section is full; only the assigned Instructor may admit from the waitlist.

Academic Records Subsystem – AcademicStandingService

Stores Transcript data and computes semester GPA and cumulative GPA after each grading period.

Enforces honor roll, academic probation, and termination conditions based on GPA thresholds.

Processes GraduationApplication records submitted by Students and reviewed by Registrars.

Manages all GradeRecord entries linked through Enrollment (chain: Student → Enrollment → GradeRecord).

Review and Complaint Subsystem – ReviewModerationService

- Accepts anonymous class reviews with star ratings and written comments.
- Scans all submitted text for taboo words: 1–2 occurrences → mask + warn; 3+ occurrences → block + 2 warnings.
- Routes Complaint records filed between users to the Registrar for resolution.
- All warnings stored individually in WarningRecord for full audit history.

AI Assistance Subsystem – VectorDBService + LLMService

- Answers user questions using a Retrieval-Augmented Generation (RAG) architecture.
- Queries VectorDBService (local knowledge base) first; if confidence meets threshold, returns answer immediately.
- If VectorDBService confidence is below threshold, falls back to LLMService (external LLM).
- LLM-generated answers are displayed with a visible disclaimer banner.
- All queries and answers logged in AIQuery with source and disclaimer fields.

1.3 Collaboration Class Diagram

The collaboration class diagram below presents the complete architecture of College0 at the system level. It is organized into four layers, read left to right, separated by a dashed system boundary. Everything to its left is an external actor; everything inside the boundary is implemented code and data.

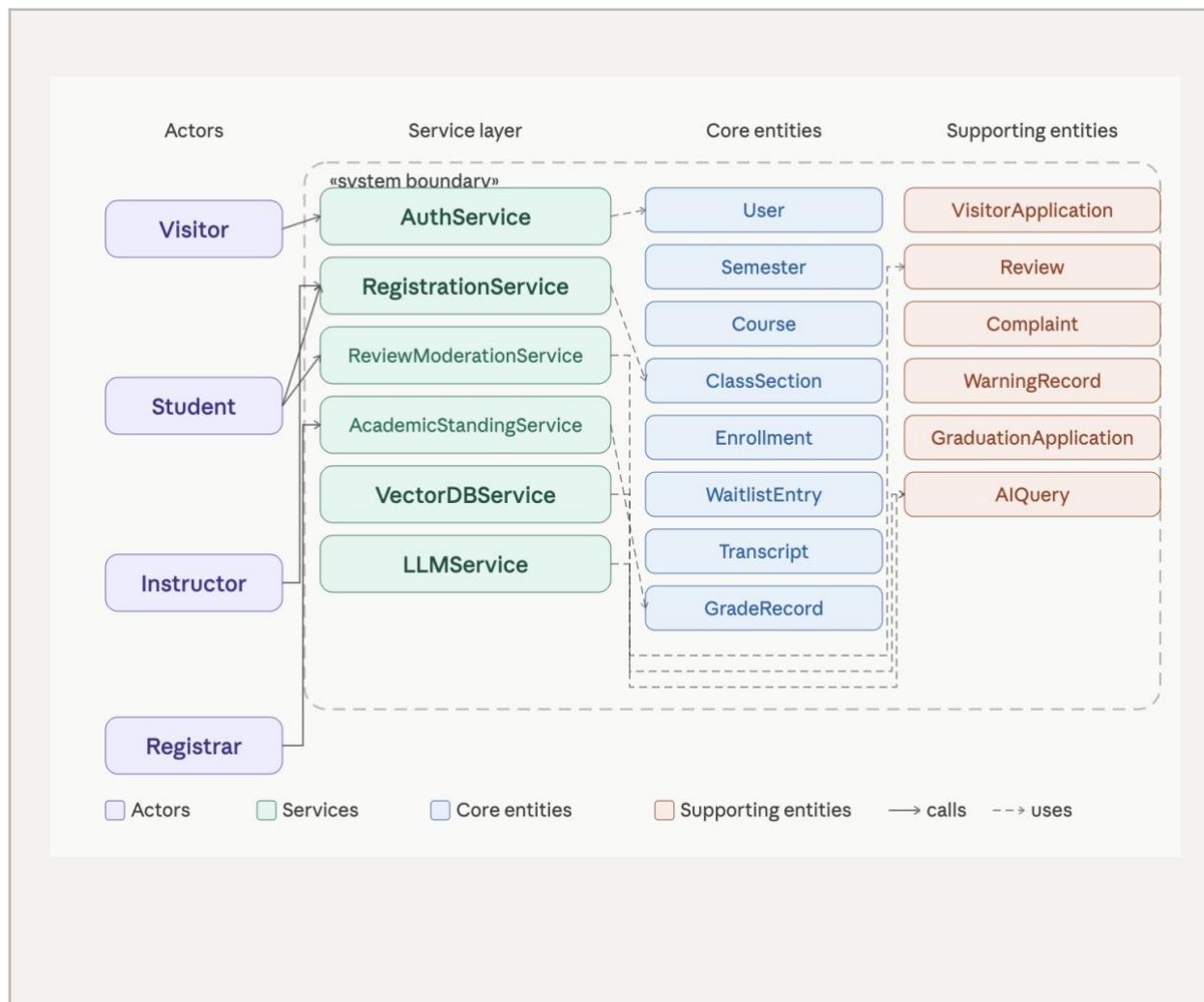


Figure 1.1 — College0 System-Level Collaboration Class Diagram

Major Classes in the Diagram

External Actors	Service Layer	Core Entities	Supporting Entities
Visitor	AuthService	User	VisitorApplication
Student	RegistrationService	Semester	Review
Instructor	ReviewModerationService	Course	Complaint
Registrar	AcademicStandingService	Class	WarningRecord
	VectorDBSer, LLMSer	Section	GraduationApplication
		Enroll-ment	AIQuery
		Waitlist-	
		Entry	
		Transcript	
		Grade	
		Record	

Main Relationships

From	Relationship	To	Service
User	parent of	Student, Instructor, Registrar	AuthService
Visitor	submits	VisitorApplication	AuthService
Registrar	reviews many	VisitorApplication	AuthService
Semester	contains many	ClassSection	RegistrationService
Course	offered as many	ClassSection	RegistrationService
Student	enrolls via	Enrollment → ClassSection	RegistrationService

Instructor	<i>teaches many</i>	ClassSection	RegistrationService
Enrollment	<i>produces one</i>	GradeRecord	AcademicStandingService
Student	<i>has one</i>	Transcript	AcademicStandingService
User	<i>receives many</i>	WarningRecord	ReviewModerationService
ClassSection	<i>has many</i>	Review	ReviewModerationService
User	<i>files / receives</i>	Complaint	ReviewModerationService
Student	<i>submits</i>	GraduationApplication	AcademicStandingService
VectorDBService	<i>queries first</i>	VectorDB → AIQuery	AI Subsystem
LLMService	<i>fallback to</i>	LLM → AIQuery	AI Subsystem

Key Design Principle

No actor communicates with the database or entity classes directly. All data access is mediated through the service layer. This ensures that every business rule — taboo word enforcement, semester phase gating, GPA thresholds, load limits, and warning counts — is applied consistently regardless of which screen or user action triggered it.

Design Note: Visitor and User

Visitor is an external, unauthenticated actor. It does **not** inherit from the User class and has no record in the users table. User is the abstract parent for Student, Instructor, and Registrar only — the three roles that require a system account. GradeRecord is linked to Enrollment, not directly to Student; the chain is: Student → Enrollment → GradeRecord.

2. All Use Cases

UC-1 Admission Processing

Actors: Visitor, Registrar

Normal Scenario:

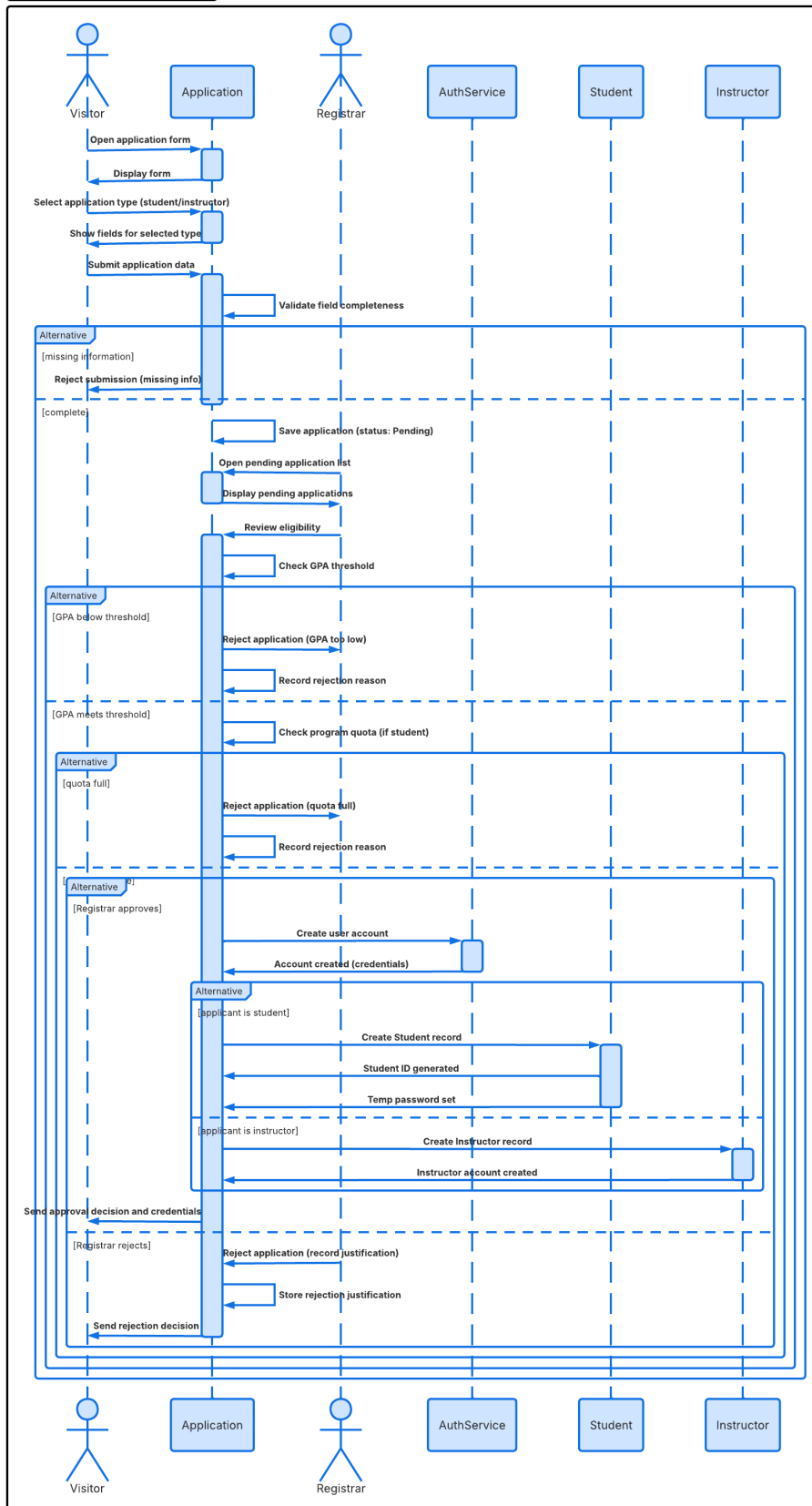
1. Visitor opens the application form.
2. Visitor selects student or instructor application type.
3. Visitor enters required data.
4. System validates field completeness.
5. Application is saved with status Pending.
6. Registrar opens the pending application list.
7. Registrar reviews eligibility.
8. If approved, the system creates a user account.
9. If applicant is a student, the system generates student ID and temporary password.
10. Applicant receives decision.

Exceptional Scenario:

- Missing information → system rejects form submission.
- GPA below required threshold → application rejected.
- Program quota full → student application rejected.
- Registrar rejects despite valid GPA/quota → justification must be recorded.

Design Classes Involved: Visitor, Application, Registrar, AuthService, Student, Instructor.

UC-1 Admission Processing - Sequence Diagram



UC-2 Semester Period Management

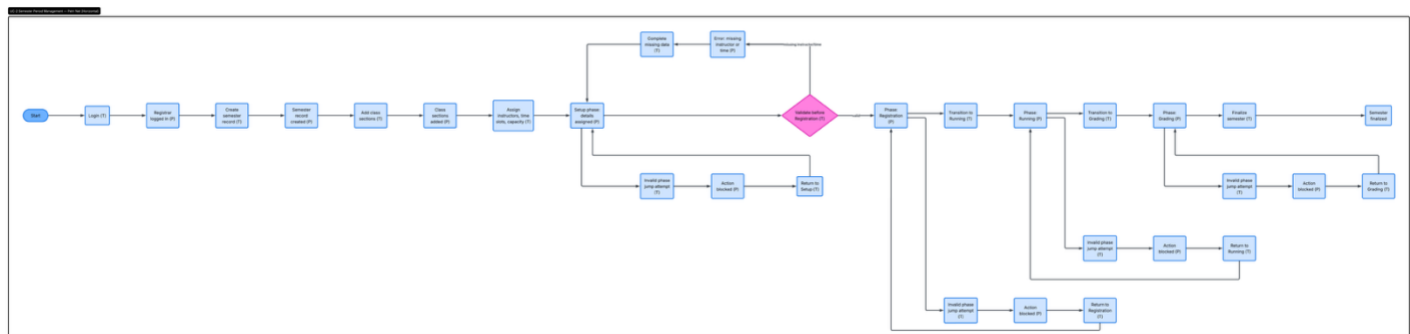
Actors: Registrar

Normal Scenario:

1. Registrar logs in.
2. Registrar creates semester record.
3. Registrar adds class sections.
4. Registrar assigns instructors, time slots, and capacity.
5. Registrar transitions phase from Setup to Registration.
6. Registrar transitions phase from Registration to Running.
7. Registrar transitions phase from Running to Grading.
8. Semester is finalized.

Exceptional Scenario:

- Registrar attempts invalid phase jump → system blocks action.
- Class missing instructor or time before registration opens → system displays error.



UC-3 Course Registration

Actors: Student

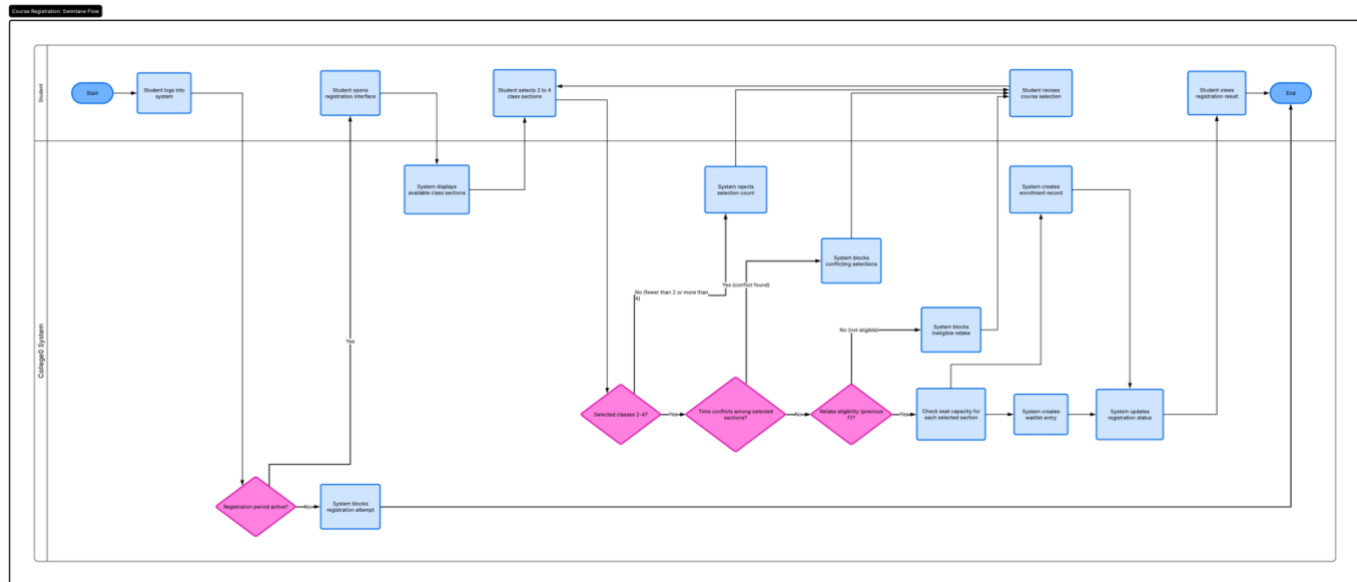
Normal Scenario:

1. Student logs in during registration period.
2. Student views available class sections.
3. Student selects 2–4 classes.
4. System checks time conflicts.
5. System checks prerequisites and retake rules.
6. System checks capacity.
7. Available classes create enrollments.
8. Full classes create waitlist entries.
9. Transcript and schedule are updated.

Exceptional Scenario:

- Student selects fewer than 2 or more than 4 classes → system blocks submission.

- Time conflict detected → system rejects conflicting classes.
- Student tries to retake without previous F → blocked.
- Registration period closed → blocked.



UC-4 Class Running and Enrollment Adjustment

Actors: Registrar, Student, Instructor

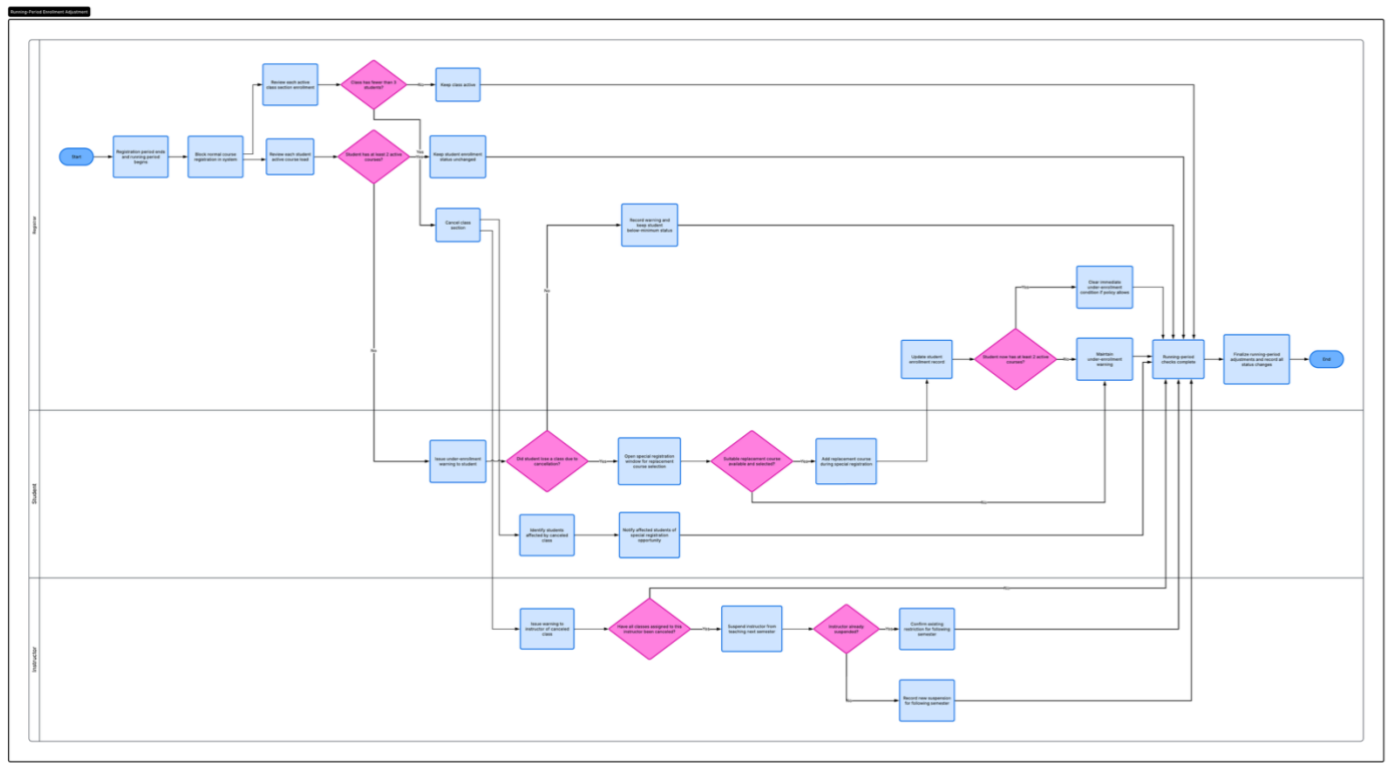
Normal Scenario:

1. Registration closes.
2. System checks student minimum course load.
3. Under-enrolled students receive warnings.
4. System finds classes with fewer than 3 students.
5. Those classes are canceled.
6. Affected students enter a short special registration period.
7. Instructors of canceled classes receive warnings.
8. If all an instructor's classes are canceled, instructor is suspended.

Exceptional Scenario:

- No replacement course available → student stays below minimum and warning remains.

- Instructor suspension status already active → next-semester teaching remains blocked.



UC-5 Class Reviews and Ratings

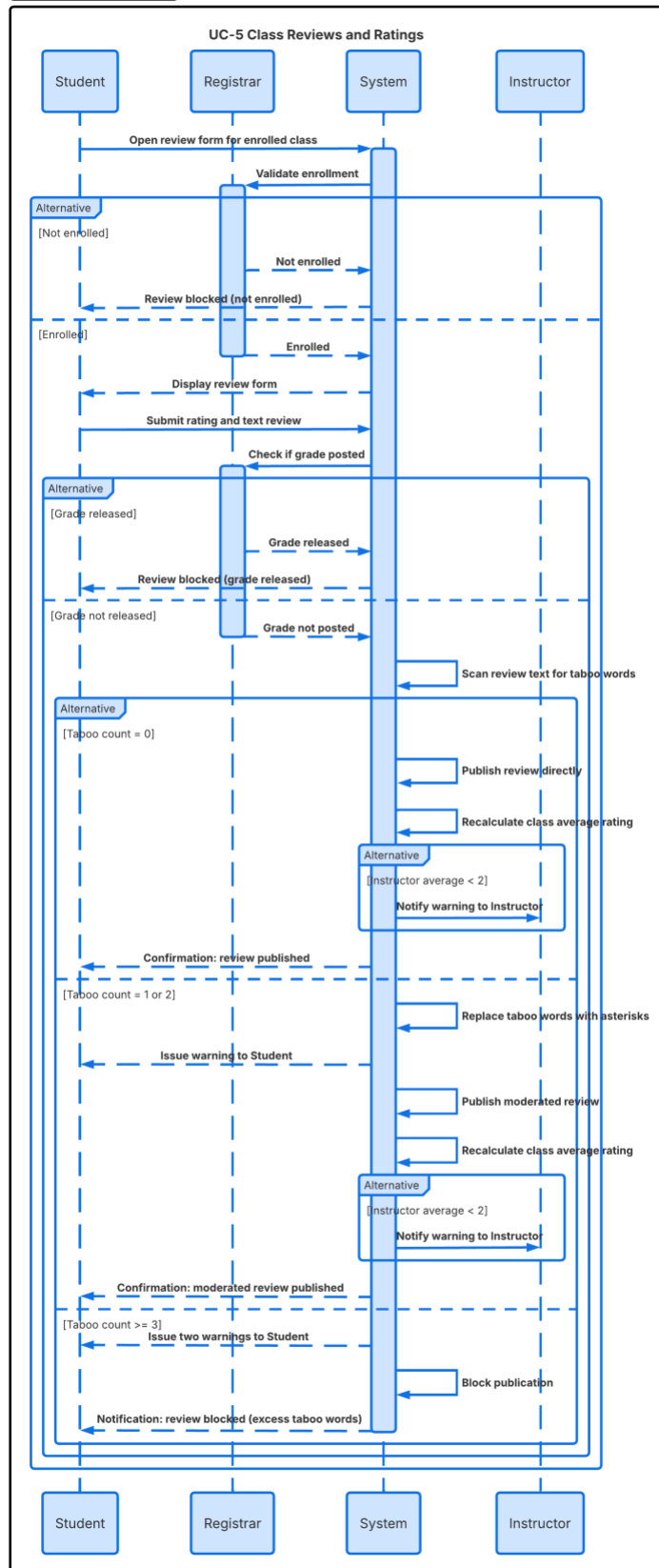
Actors: Student, Registrar, Instructor

Normal Scenario:

1. Student opens review form for enrolled class.
2. Student enters rating and text review.
3. System checks if grade has already been posted.
4. System scans review text for taboo words.
5. If taboo count is 0 → publish directly.
6. If taboo count is 1 or 2 → replace taboo words with asterisks, issue warning, publish moderated review.
7. If taboo count is 3 or more → block publication, issue two warnings.
8. System recalculates class average rating.
9. If instructor average drops below 2, instructor receives warning.

Exceptional Scenario:

- Student not enrolled in class → review blocked.
- Student tries to review after grade release → review blocked.



UC-6 Grading and Academic Standing

Actors: Instructor, Registrar

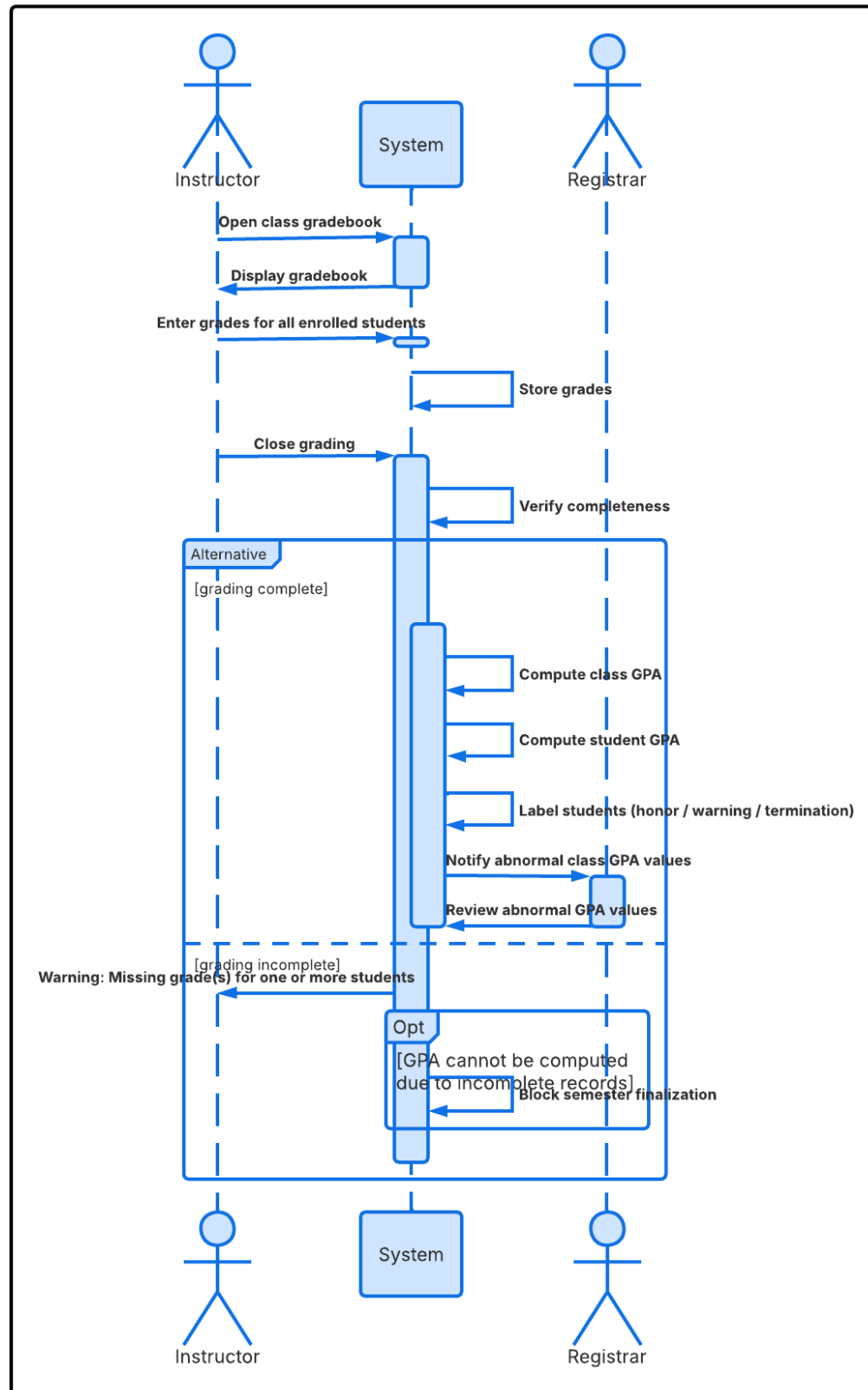
Normal Scenario:

1. Instructor opens class gradebook.
2. Instructor enters grades for all enrolled students.
3. System stores grades.
4. After grading closes, system verifies completeness.
5. Incomplete grading triggers instructor warning.
6. System computes class GPA and student GPA.
7. Students are labeled as honor roll, warning, or termination status based on rules.
8. Registrar reviews abnormal class GPA values.

Exceptional Scenario:

- Grade missing for one or more students → instructor warning.
- GPA data cannot be computed because of incomplete records → semester finalization blocked.

UC-6 Grading & Academic Standing — Sequence Diagram



UC-7 Graduation Processing

Actors: Student, Registrar

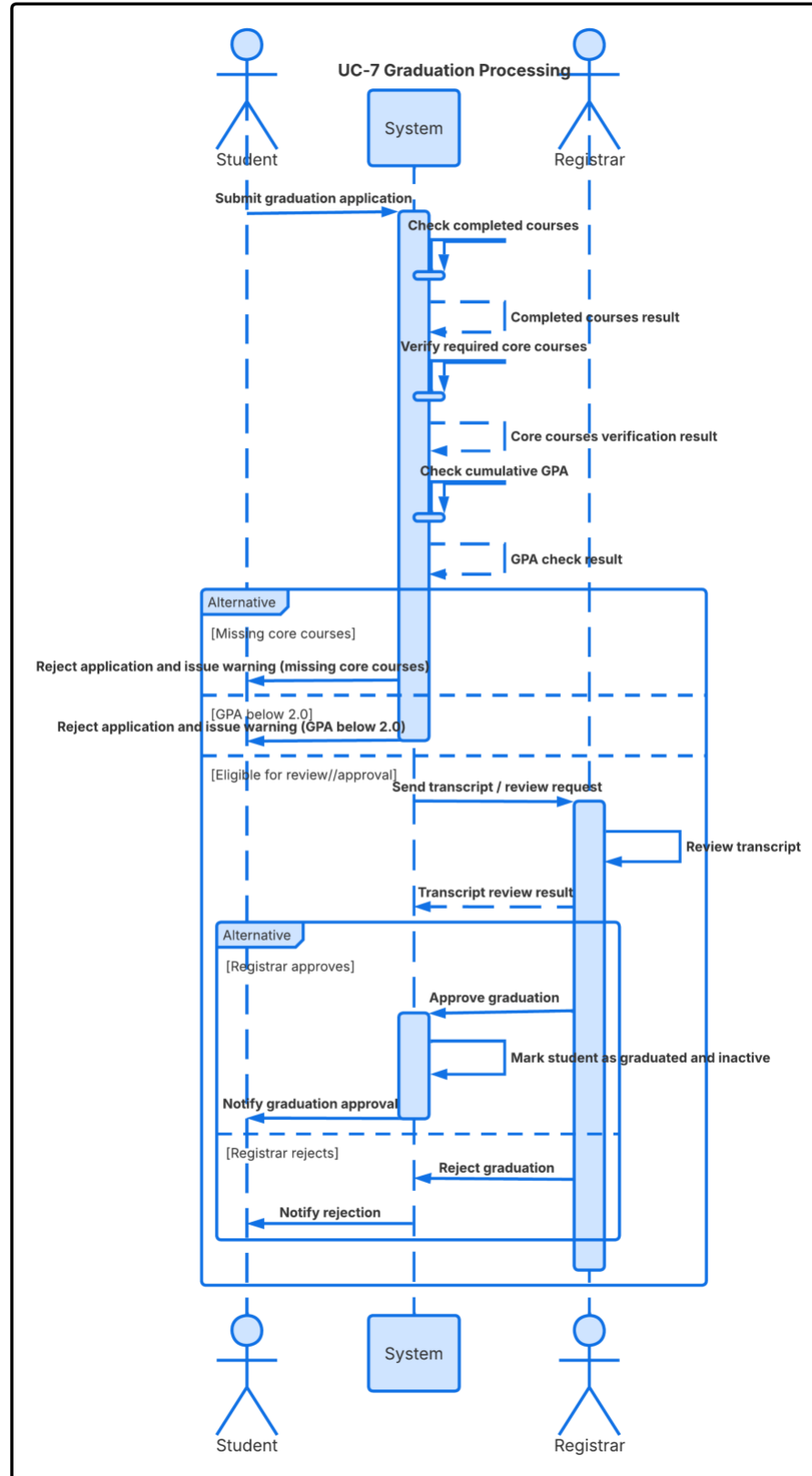
Normal Scenario:

1. Student submits graduation application.
2. System checks completed courses.
3. System verifies required core courses.
4. System checks cumulative GPA.
5. Registrar reviews transcript.
6. If eligible, registrar approves graduation.
7. System marks student as graduated and inactive.

Exceptional Scenario:

- Missing core courses → reject application and issue warning.
- GPA below 2.0 → reject application and issue warning.

UC-7 Graduation Processing



UC-8 Complaint and Discipline Processing

Actors: Student, Instructor, Registrar

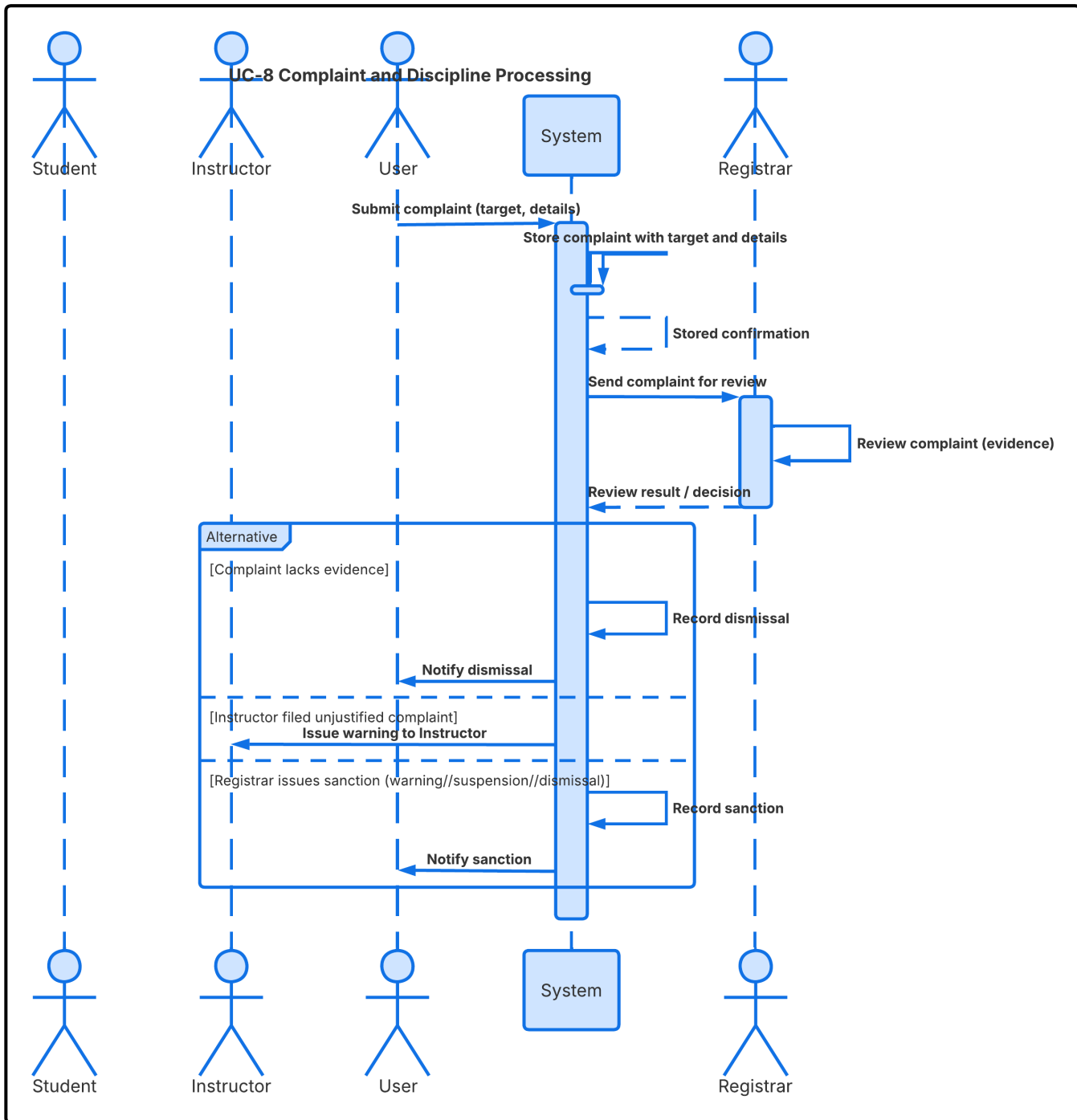
Normal Scenario:

1. User submits complaint.
2. System stores complaint with target user and details.
3. Registrar reviews complaint.
4. Registrar records decision.
5. System issues warning, suspension, or dismissal if needed.

Exceptional Scenario:

- Complaint lacks evidence → registrar dismisses complaint.
- Instructor files unjustified complaint → instructor gets warning.

UC-8 Complaint and Discipline Processing



UC-9 Role-Based User Access

Actors: Student, Instructor, Registrar

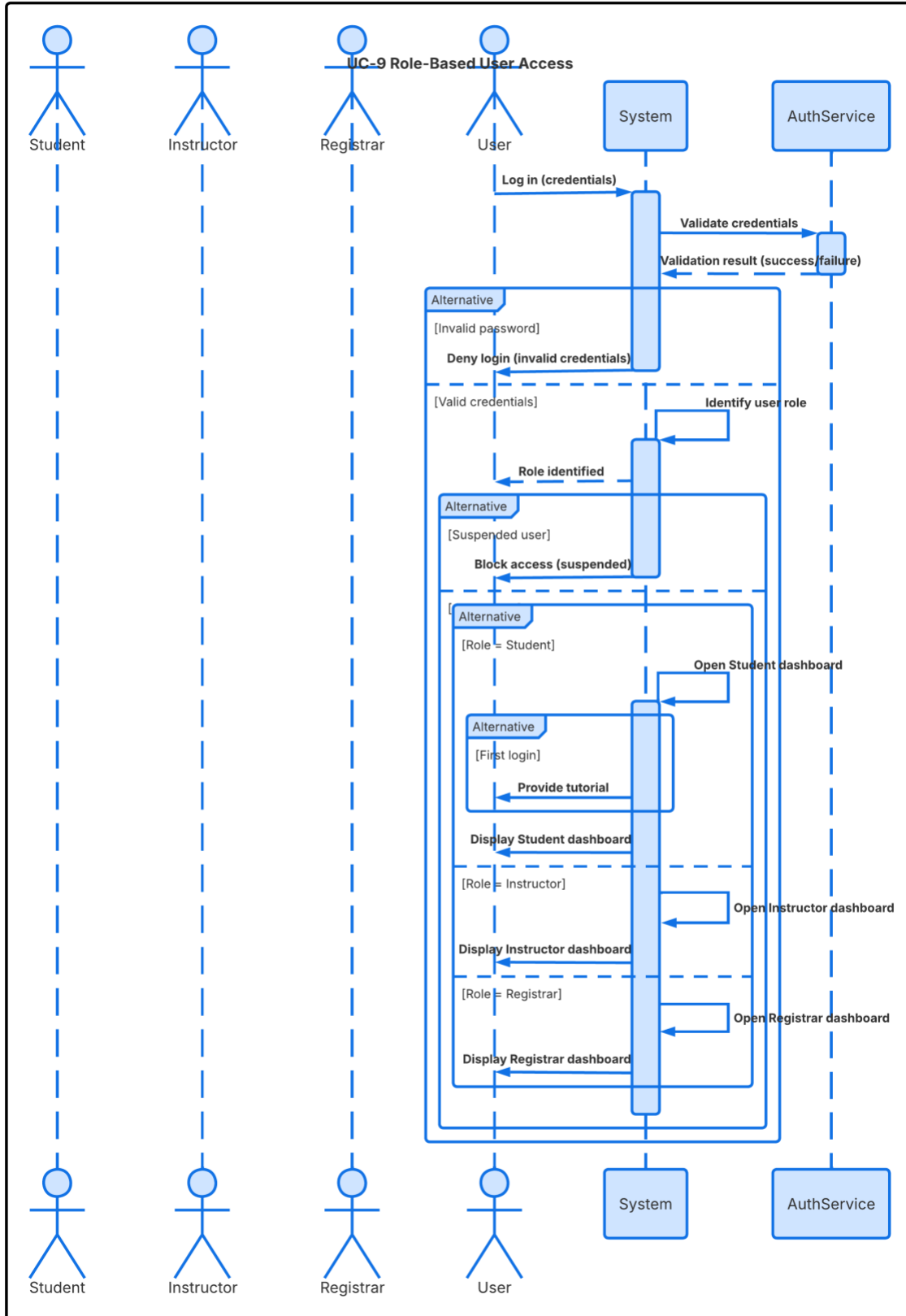
Normal Scenario:

1. User logs in.
2. AuthService validates credentials.
3. System identifies role.
4. System opens correct dashboard.
5. Student may receive tutorial if first login.

Exceptional Scenario:

- Invalid password → login denied.
- Suspended user tries to access restricted features → blocked.

UC-9 Role-Based User Access



UC-10 AI-Enabled Assistance

Actors: Visitor, Student, Instructor

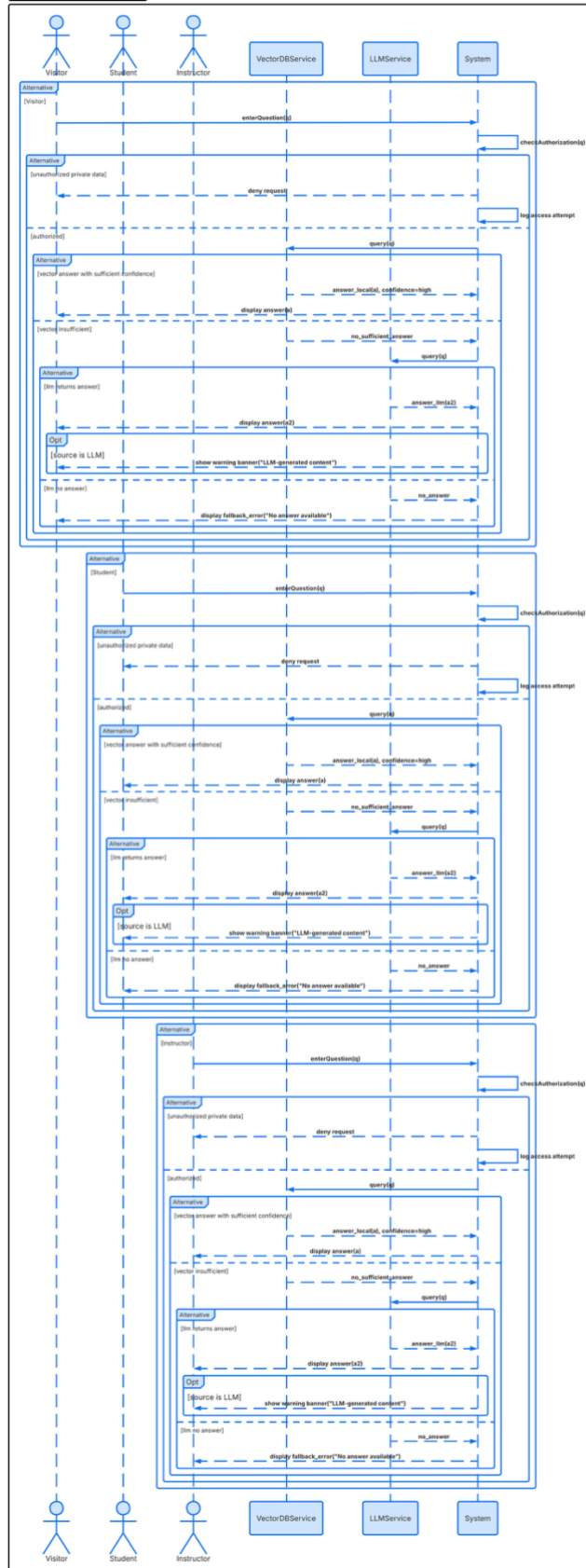
Normal Scenario:

1. User enters question.
2. System sends query to VectorDBService.
3. If answer confidence is sufficient, return local answer.
4. Otherwise send query to LLMService.
5. System displays answer.
6. If source is LLM, system shows warning banner.

Exceptional Scenario:

- No answer from VectorDB and no response from LLM → display fallback error message.
- User asks for unauthorized private data → system denies and logs access attempt.

UC-19 AI-Enabled Assistance - Sequence Diagram



UC-11 Creative Feature: Academic Progress Tracker

Actors: Student

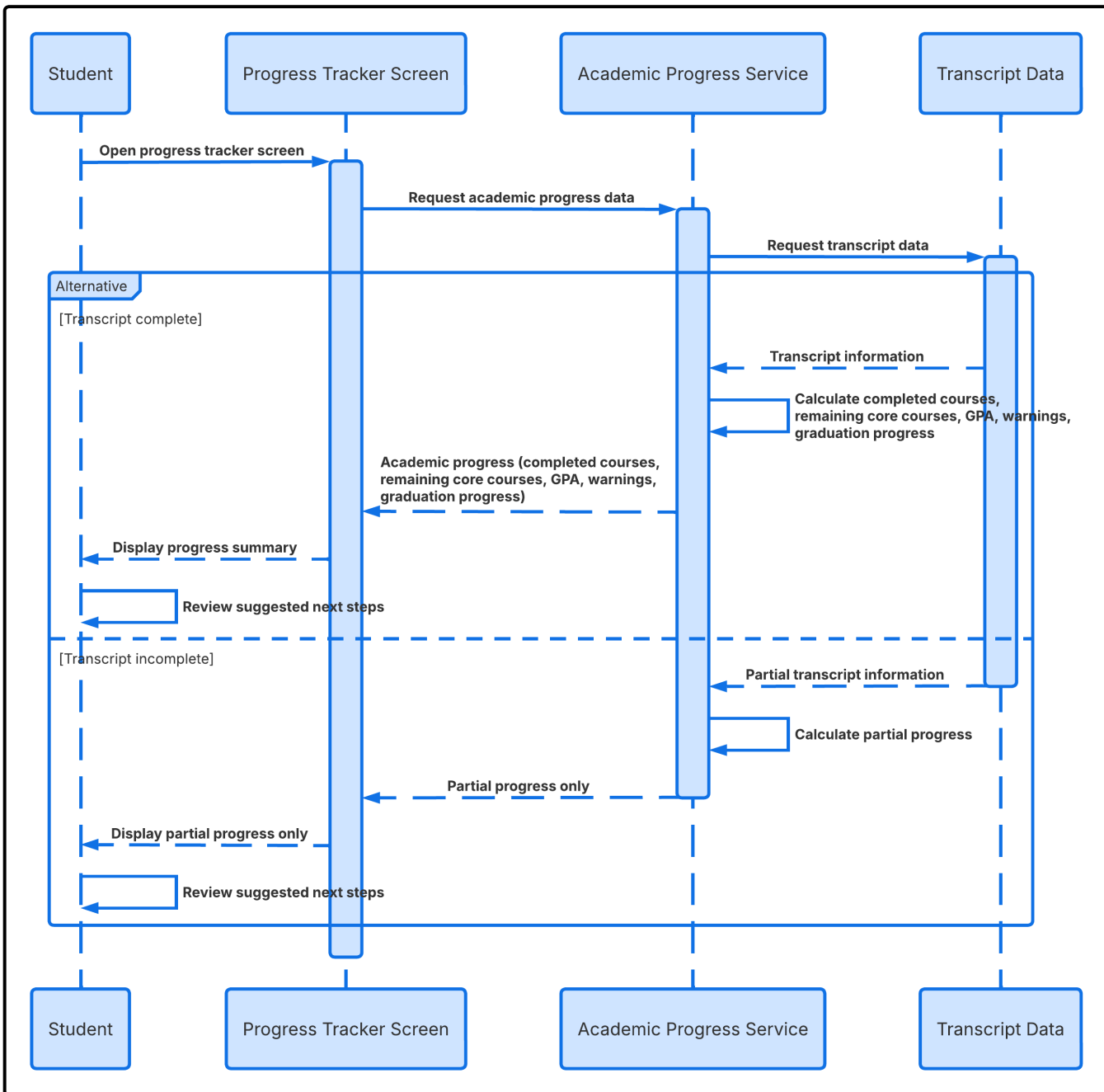
Normal Scenario:

1. Student opens progress tracker screen.
2. System shows completed courses, remaining core courses, GPA, warnings, and graduation progress.
3. Student reviews suggested next steps.

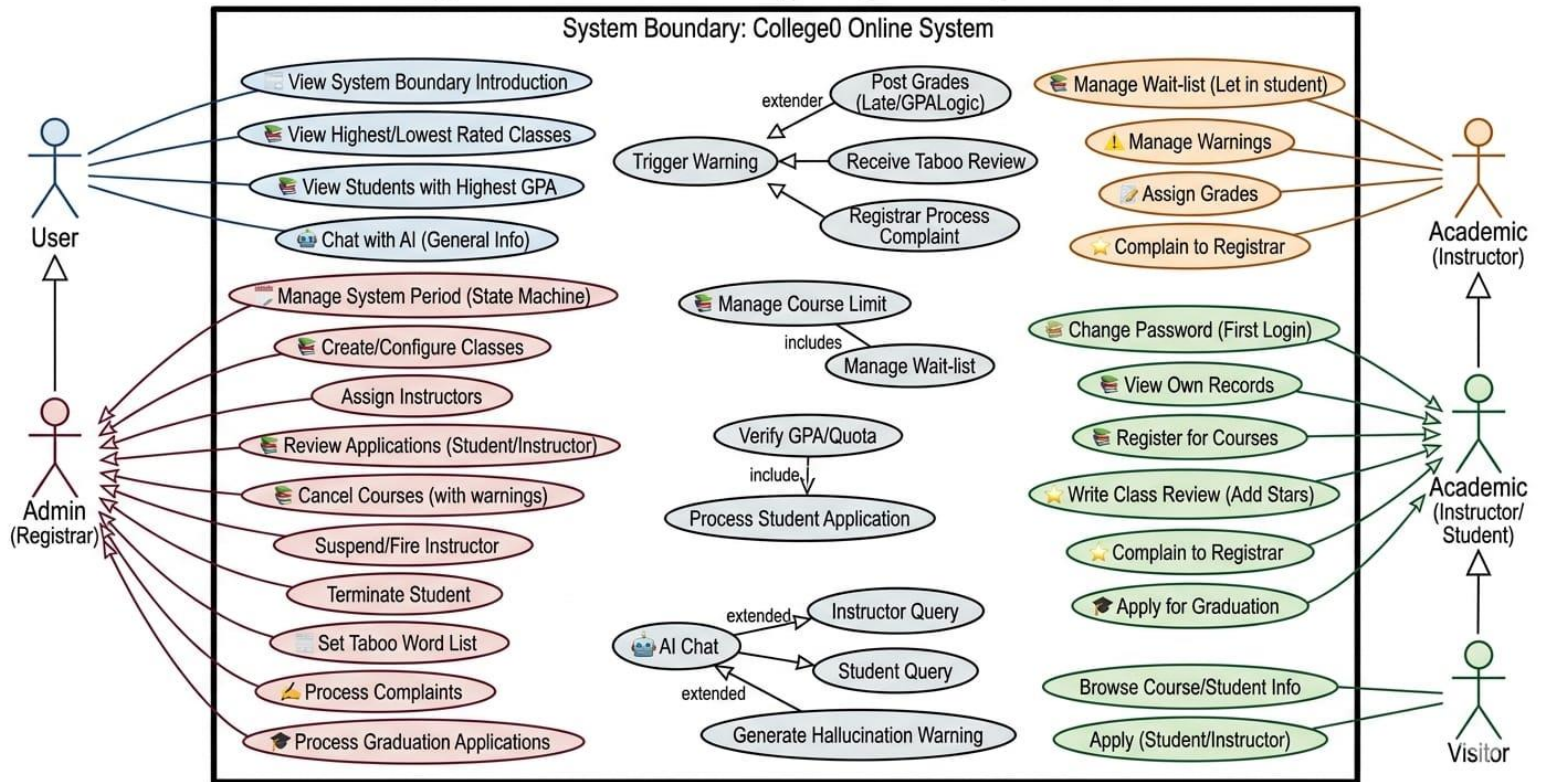
Exceptional Scenario:

- Transcript incomplete → screen shows partial progress only.

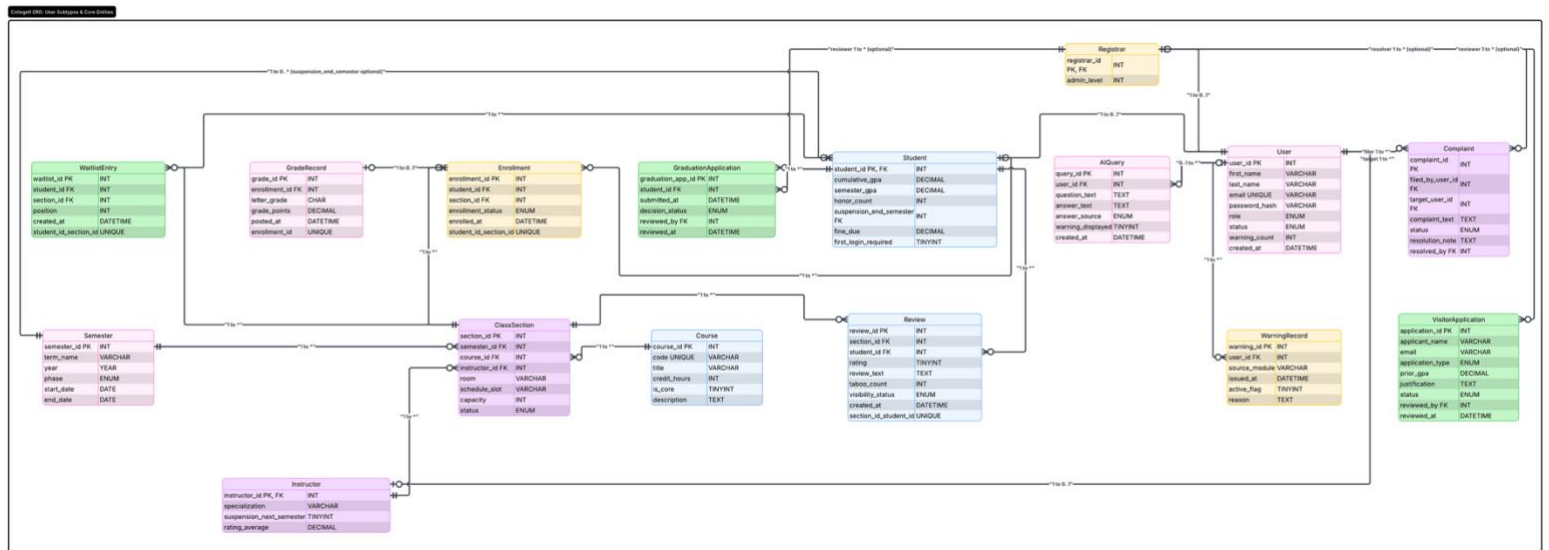
UC-11: Academic Progress Tracker - Sequence Diagram



College0: AI-Enabled Online College Program Management System



Use Case Diagram



Entity Relationship Diagram

4. Detailed Design

4.1 Design Approach

The system follows an object-oriented structure with service classes controlling business logic. GUI classes call service-layer methods. Service methods validate rules and then update persistent entities.

Main design layers:

- **Presentation Layer** – GUI screens and forms
- **Service Layer** – admission, registration, review moderation, GPA, AI logic
- **Data Layer** – entities, repositories, and database/file persistence

4.2 Pseudocode for Major Methods

1. Front-End Layer (User Interface Logic)

This section is responsible for collecting data from the user and interacting with the backend.

Method: handleLogin(userID, password)

Input: userID, password

Output: success (redirect) or login failure message

IF inputs are empty

 RETURN validation error

CALL backend API for authentication

IF authentication is successful

 SAVE user session information

 REDIRECT to appropriate dashboard

ELSE

 RETURN login failure message

Method: handleApply(appFormData)

Input: appFormData

Output: confirmation or error message

IF appFormData is invalid

 RETURN validation error

SEND appFormData to backend API

IF submission is successful

 RETURN confirmation message

ELSE

 RETURN error message

Method: handleAskAI(userQuestion)

Input: userQuestion

Output: AI answer or error message

IF userQuestion is empty

 RETURN error

SET loading indicator = Visible

CALL backend API with userQuestion

IF response is successful

 DISPLAY AI answer

ELSE

 DISPLAY error message

Method: loadDashboard(userRole)

Input: userRole

Output: Rendered dashboard or error state

REQUEST role-specific data from backend API

IF data is received

 RENDER dashboard components

ELSE

 DISPLAY loading or error state

2. Back-End Layer (Business Logic & Core Services)

The system's core functionalities and business rules are handled here.

Method: login(email, password)

Input: email, password

Output: user role or dashboard route

LOAD user by email

IF user not found OR password invalid

 RETURN authentication error

IF user first_login_required = true

 FORCE password change

RETURN user role and dashboard route

Method: submitApplication(appData)

Input: applicant data

Output: success/failure message

IF required fields are missing

 RETURN error

CREATE Application object

SET status = Pending

SAVE application

RETURN success

Method: reviewApplication(applicationId, decision, justification)

Input: applicationId, decision, justification

Output: approved/rejected result

LOAD application

IF application not found

 RETURN error

IF decision = APPROVE AND application.type = STUDENT

 IF application.gpa <= 3.0 OR quota full

 RETURN rejection error

IF decision = REJECT AND application.type = STUDENT AND application.gpa > 3.0 AND quota not full

 REQUIRE justification

UPDATE application status

IF approved

 CREATE user account

 IF student

 GENERATE student ID and temporary password

RETURN result

Method: openRegistration(semesterId)

Input: semesterId

Output: success/error message

LOAD semester

IF current phase != Setup

 RETURN error

SET phase = Registration

SAVE semester

RETURN success

Method: registerCourses(studentId, selectedSections)

Input: studentId, selectedSections

Output: registration summary

```
IF current semester phase != Registration
    RETURN error
IF count(selectedSections) < 2 OR count(selectedSections) > 4
    RETURN error
FOR each section in selectedSections
    CHECK time conflict
    CHECK retake rule
    IF invalid
        RETURN error
FOR each section in selectedSections
    IF section has space
        CREATE enrollment
    ELSE
        CREATE waitlist entry
RETURN registration summary
```

Method: admitFromWaitlist(instructorId, sectionId, studentId)

Input: instructorId, sectionId, studentId

Output: success/error message

```
VERIFY instructor teaches section
VERIFY student is on waitlist
IF section full
    RETURN error
REMOVE waitlist entry
CREATE enrollment
RETURN success
```

Method: runEnrollmentAdjustment(semesterId)

Input: semesterId

Output: adjustment summary

```
LOAD all active sections in semester
FOR each student
    IF enrolled course count < 2
        ISSUE warning
FOR each section
    IF enrolled student count < 3
        CANCEL section
        WARN instructor
```

```
    MARK affected students for special registration
FOR each instructor
    IF all assigned sections canceled
        SET suspension_next_semester = true
RETURN summary
```

Method: submitReview(studentId, sectionId, rating, text)

Input: studentId, sectionId, rating, text

Output: review status

```
VERIFY student enrolled in section
VERIFY final grade not posted
tabooCount = countTabooWords(text)
IF tabooCount = 0
    status = Published
ELSE IF tabooCount <= 2
    text = censorTabooWords(text)
    ISSUE 1 warning to student
    status = PublishedModerated
ELSE
    ISSUE 2 warnings to student
    status = Blocked
SAVE review
UPDATE class rating average if review visible
CHECK instructor average rating
IF instructor average < 2
    ISSUE warning to instructor
RETURN status
```

Method: assignGrades(instructorId, sectionId, gradeList)

Input: instructorId, sectionId, gradeList

Output: success/error message

```
VERIFY instructor teaches section
FOR each grade item in gradeList
    FIND enrollment
    SAVE grade record
RETURN success
```

Method: finalizeGrading(semesterId)

Input: semesterId

Output: summary report

```

FOR each section in semester
  IF not all students graded
    ISSUE warning to instructor
  classGpa = calculateClassGPA(section)
  IF classGpa > 3.5 OR classGpa < 2.5
    FLAG for registrar review
FOR each student
  gpa = calculateStudentGPA(student)
  UPDATE cumulative GPA
  IF gpa < 2.0 OR failedSameCourseTwice(student)
    SET status = Terminated
  ELSE IF gpa >= 2.0 AND gpa <= 2.25
    ISSUE warning
  IF qualifiesHonorRoll(student)
    ADD honor
  REMOVE one warning if possible
RETURN summary

```

Method: processGraduation(studentId)

Input: studentId

Output: approved/rejected status

```

LOAD transcript
IF completedCourses < 8 OR missing core courses OR cumulative GPA <= 2.0
  ISSUE reckless graduation warning
  RETURN rejected
APPROVE graduation
SET student status = Graduated
RETURN approved

```

Method: getProgressReport(studentId)

Input: studentId

Output: progress summary object

```

LOAD transcript
completed = count passed courses
coreCompleted = count passed core courses
remaining = graduation requirement - completed
gpa = cumulative GPA
warnings = active warnings
RETURN progress summary object

```

3. API & External Service Layer

This is the part that manages AI services and complaints.

Method: answerAIQuery(userId, question)

Input: userId, question

Output: AI answer (local or LLM)

localAnswer = VectorDBService.search(question, userScope)

IF localAnswer confidence sufficient

 SAVE AIQuery with source = Local

 RETURN localAnswer

llmAnswer = LLMService.ask(question, userScope)

IF llmAnswer unavailable

 RETURN fallback error

SAVE AIQuery with source = LLM and warning_displayed = true

RETURN llmAnswer with hallucination warning

Method: fileComplaint(filedBy, targetUser, text)

Input: filedBy, targetUser, text

Output: success message

VERIFY filer is allowed to submit complaint

CREATE complaint record

SET status = Pending

SAVE complaint

RETURN success

Method: resolveComplaint(complaintId, decision, penalty)

Input: complaintId, decision, penalty

Output: success/error message

LOAD complaint

IF complaint not found RETURN error

SET complaint status = Resolved

SAVE resolution note

IF penalty required

 ISSUE warning or suspension to target user

IF complaint unjustified AND filer is instructor

 ISSUE warning to filer

RETURN success

4. Database & Utility Layer (Helper Methods)

These are small utility components that handle calculations and database information

Method: calculateGPA(studentId)

Input: studentId

Output: GPA (number)

```
LOAD all completed enrollments for student
SET totalPoints = 0, totalCredits = 0
FOR each enrollment
    GET pointValue from grade
    totalPoints += (pointValue * courseCredits)
    totalCredits += courseCredits
IF totalCredits == 0 RETURN 0.0
RETURN totalPoints / totalCredits
```

Method: countTabooWords(text)

Input: text

Output: count (integer)

```
LOAD tabooWords from DB
SET count = 0
FOR each word in text (lowercase)
    IF word in tabooWords SET count++
RETURN count
```

Method: censorTabooWords(text)

Input: text

Output: censored text

```
LOAD list of tabooWords
FOR each word in tabooWords
    REPLACE word in text with "*****"
RETURN censored text
```

Method: issueWarning(userId)

Input: userId

Output: user.warnings count

```
LOAD user record
SET user.warnings = user.warnings + 1
```

```
IF user.warnings >= 3
  CALL suspendUser(userId)
SAVE user record
RETURN user.warnings
```

Method: suspendUser(userId)

Input: userId
Output: suspension status message

```
LOAD user record
SET user.status = Suspended, user.suspensionDate = current date
SAVE user record
NOTIFY user via email
RETURN "User suspended successfully"
```

Method: failedSameCourseTwice(studentId)

Input: studentId
Output: Boolean (true/false)

```
LOAD student transcript
FILTER transcript for grades = 'F'
GROUP failed courses by courseCode
FOR each group
  IF count(group) >= 2 RETURN true
RETURN false
```

5. System Screens

The following major frontend screens were developed for College0.

5.1 Login Screen

The login screen allows users to enter their user ID and password in order to access the portal. It includes a sign-in form, demo credential guidance, and error handling for invalid login attempts.

This interface allows users to authenticate and access the academic portal.

5.2 Home / Public Page

The home page presents an overview of the academic portal. It includes campus summary information, highlighted academic statistics, top-rated courses, top GPA students, and the course

catalogue. Figure 2. Public home page of College0. This screen presents an overview of the portal, including academic statistics, featured courses, and the course catalogue.

5.3 Apply Screen

The application screen allows visitors to submit an application to join the system as a student or instructor. It includes form fields for identity, email, role selection, GPA, and an optional statement.

Figure 3. Application screen. This page allows visitors to submit an application to the system.

5.4 AI Assistant Screen

The AI Assistant page provides a user interface where users can ask academic or portal-related questions. The frontend supports text input, quick question buttons, and response display.

Figure 4. AI Assistant interface. This page allows users to submit questions and interact with the assistant interface.

5.5 Student Dashboard

The student dashboard is shown after successful login and displays personalized academic information such as GPA, warning count, enrolled courses, and completed courses.

Figure 5. Student dashboard. This dashboard provides a personalized academic overview for the logged-in student.

5.6 My Courses Screen

The My Courses page displays the courses currently enrolled by the student and related semester information. Figure 6. My Courses screen. This page displays the student's enrolled courses and semester registration details.

5.7 Transcript Screen

The transcript page presents the academic record of the student, including cumulative GPA, warning count, honor count, and completed courses with grades. Figure 7. Academic Transcript

screen. This page shows the student's academic record and performance summary.

5.8 Reviews & More Screen

The Reviews & More page provides access to review submission, graduation-related features, and complaint-related actions. It supports tabbed interaction and form input. Figure 8. Reviews & More screen. This page provides access to review, graduation, and complaint functions.

Sequence Flow

1. Login Sequence

User → Login Screen → Backend → Frontend → Dashboard

- 1) User enters login credentials.
- 2) Frontend sends authentication request to backend.
- 3) Backend checks the submitted credentials.
- 4) Backend returns success or failure.
- 5) Frontend shows an error or redirects to the dashboard.

2. Application Sequence

Visitor → Apply Screen → Backend → Frontend

- 1) Visitor fills out the application form.
- 2) Frontend validates user input.
- 3) Frontend sends application data to backend.
- 4) Backend stores the application.
- 5) Frontend displays confirmation or error.

3. AI Assistant Sequence

User → AI Assistant Screen → Backend → Frontend

- 1) User enters a question.

- 2) Frontend sends the question to the backend.
- 3) Backend processes the request.
- 4) Backend returns a response.
- 5) Frontend displays the returned answer.

6. Memos of Group Meetings and Possible Concerns of Teamwork

Group Members: Aung Latt, Karim Khalil, Zihad Chowdhury, Ferdosi Hossain, Kazi Mehzabin

Meeting 1

Date: Sunday, March 22, 2026

Time: 12:00 PM

Platform: Discord (Study Lounge)

Project Phase: Phase I

Task Distribution (Phase I)

Introduction: Ferdosi Hossain

Use Case Model: Kazi Mehzabin

Specific Requirements & Supporting Information:

Shared among Aung Latt, Zihad Chowdhury, and Karim Khalil

Discussion Summary:

This was the first meeting where the team discussed the overall project idea and structure. The goal was to understand the requirements and plan Phase I. The team talked about the system design, including frontend, backend, database, and API. We also discussed the framework, tools, and resources we might use. Tasks were divided among members based on the report sections.

Decisions Made

The project idea and structure were finalized.

Work was divided among team members.

A general development approach was agreed on.

Concerns / Notes

Some parts of the system design were still general and needed more detail later.

Coordination between frontend, backend, and database would be important.

Meeting 2

Date: Monday, April 20, 2026

Time: 9:00 PM – 9:40 PM

Platform: Discord (Study Lounge)

Project Phase: Phase II

Task Distribution (Phase II)

Introduction & system overview: Zihad Chowdhury

Use cases (normal and exceptional scenarios, Petri nets): Ferdosi Hossain

E-R diagram (attributes and keys): Aung Latt

Detailed design + system screens: Karim Khalil

Meeting memos & teamwork section: Kazi Mehzabin

GitHub repository (Section 7): Whole group

Discussion Summary

The team met to organize Phase II and divide all report tasks. Each member was assigned a clear section. We also set up GitHub with separate branches so everyone can work independently.

The system design was discussed again, including backend, frontend, database, API, and AI chatbot. The plan is to build each part separately and then connect them.

Work Progress

Front-end work has started. Backend development is also in progress. Database design and backend structure are being planned. The AI chatbot and API system were discussed, including how they will connect with the backend.

Decisions Made

Each member will work on their own GitHub branch.

Work will be done in parallel (frontend, backend, database).

API and chatbot will be part of the backend system.

Concerns / Notes

Frontend and backend must be well coordinated during integration.

AI chatbot and API design still need more detail.

Database design must match backend requirements.

7. <https://github.com/AungLatt2001/Software-Engineering-Project-College0->